

Cours Programmation Orientée Objet : JAVA 1

Enseignant : **Malek BEN SALEM**

mal.bensalem@gmail.com

3^{ème} année Génie Informatique

PRÉSENTATION DU COURS

❑ Objectif général :

Ce cours vise à faire acquérir à l'apprenant :

- Le principe et les concepts de la programmation Orienté Objet,
- La syntaxe de base du langage Java
- Le concept orienté objet avec Java
- Maîtriser Les notions de paquetages, d'héritage, des classes anonymes, des classes abstraites, des interfaces et des exceptions.
- Maîtriser la programmation Orienté Objet avec la pratique

Références du Cours

- **Livre** « Java : Les fondements du langage » de Thierry RICHARD, édité par **Eni Editions** en **Mars 2022**.
- **Livre** « Apprendre la programmation Orienté Objet avec le langage Java » de Luc GERVAIS, édité par **Eni Editions** en **Août 2020**.
- **Livre** « Java et Eclipse : Développez une application avec Java et Eclipse (2e édition) » de Frédéric DÉLÉCHAMP, édité par **Eni Editions** en **Mars 2018**.

Plan du Cours

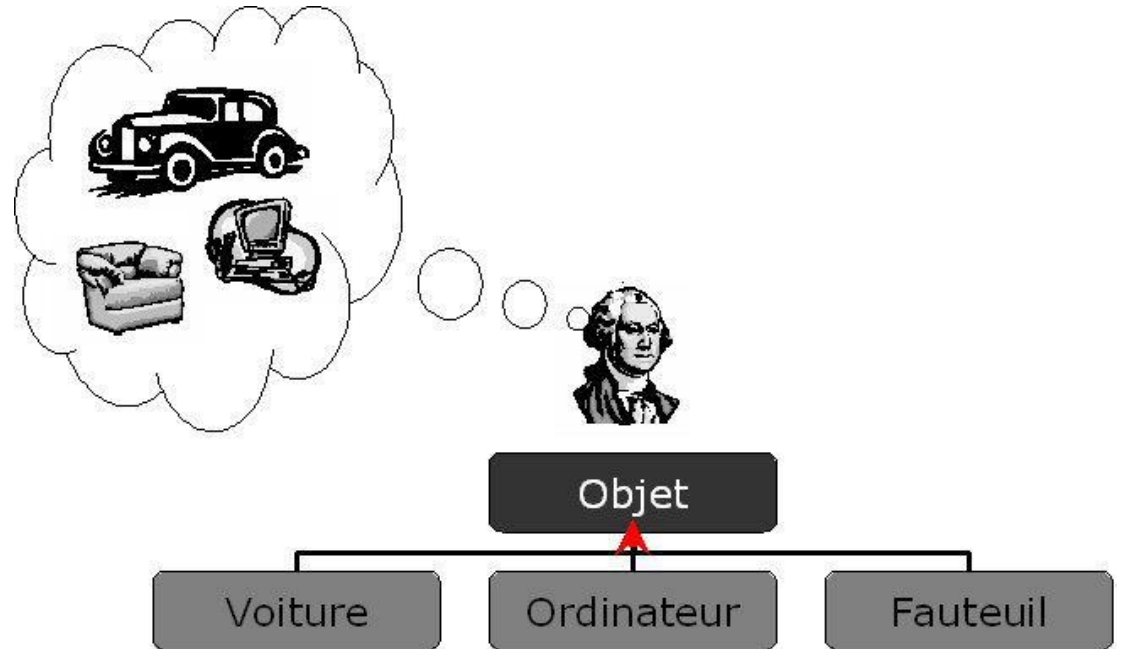
1. Chapitre 1 : Généralité et syntaxe de base
2. **Chapitre 2 : Concept Objet en Java**
3. Chapitre 3 : Héritage
4. Chapitre 4 : Classes abstraites et Interfaces
5. Chapitre 5 : Les exceptions

Chapitre 2: Concept Objet en Java

1. Les bienfaits de l'abstraction
2. Approche Objet : Classe et Objet
3. Encapsulation des objets
4. Modificateur de visibilité
5. Constructeur d'une classe
6. Création d'une instance
7. Destruction d'un objet
8. Comparaisons d'instances
9. Les opérateurs sur les références

1. Les bienfaits de l'abstraction

- **L'abstraction** dans **l'approche objet** permet la représentation des **entités du monde réel** sous forme d'**entités informatique** de la manière la **plus naturelle**.
- Etablir une association entre le **modèle du problème** à résoudre et le **modèle de la machine** :



1. Les bienfaits de l'abstraction

- **L'abstraction** est une représentation des éléments du monde réel « **objets réels** » dans l'espace problème (la machine) en tant qu'« **objets informatiques** ».
- L'idée est d'adapter le programme à l'esprit du problème réel en ajoutant de nouveaux types « **objets** ».

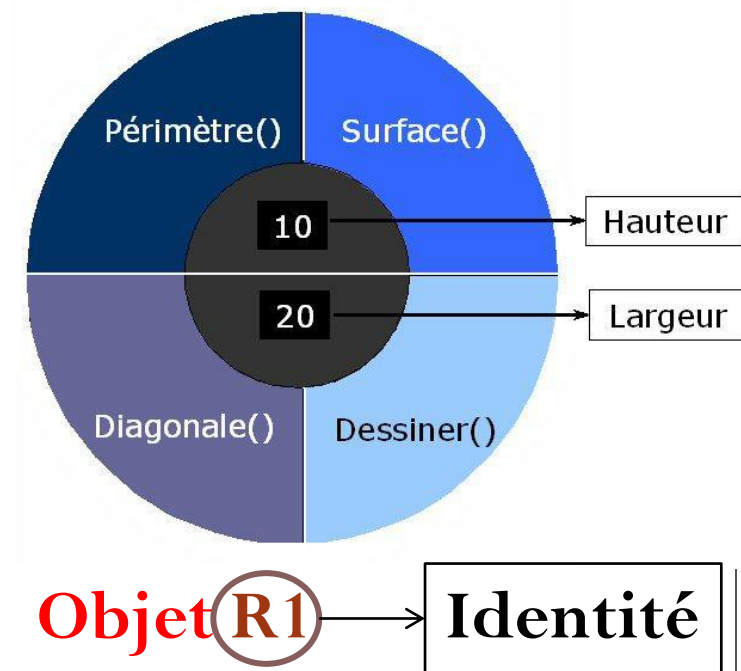
2. Approche Objet

- Un **objet** est semblable à une **variable** améliorée:
 - elle stocke des données qui décrivent son « **état** »;
 - mais qui possède aussi un ensemble de fonctions ou **méthodes** «**comportement**», pour répondre aux « **requêtes** » des utilisateurs.
- L'ensemble des services (**méthodes**) proposées par un **objet** est appelé **l'interface** de cet **objet**.
- Un **objet** est **encapsulé** par son **interface**:
 - la seule manière d'interagir (demander un service) avec cet **objet** est d'**invoker** une des **méthodes** de son **interface**.

2. Approche Objet

- Exemple : **Objet Rectangle** R1

- R1 est un **objet Rectangle** de hauteur 10 de largeur 20, ayant une interface qui permet de :
 - calculer le périmètre du rectangle,
 - sa surface,
 - La diagonale
 - et un dernier service pour dessiner le rectangle.
- Pour utiliser l'un de ces services il suffit **d'invoquer la méthode** correspondante.

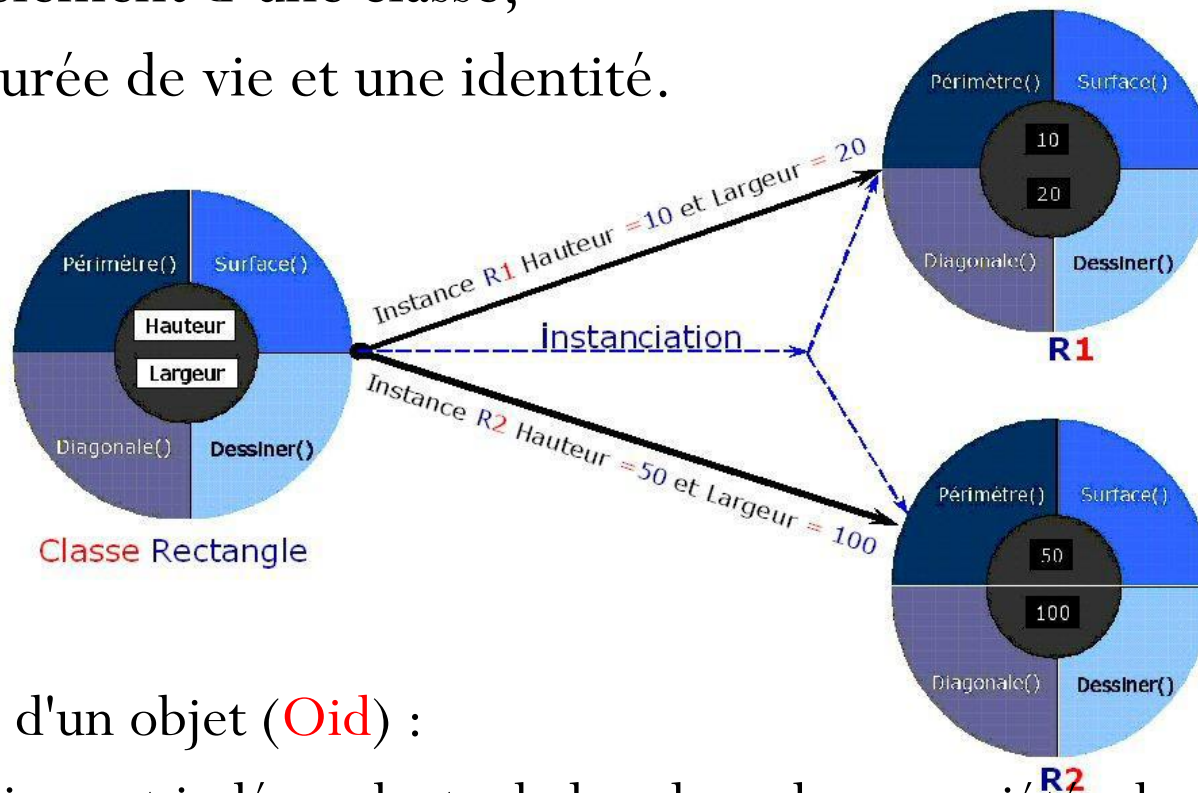


2. Approche Objet

- Une **classe** est un moule pour fabriquer des objets de **même structure** et de **même comportement**.
 - En programmation, une **classe** apparaît donc comme un nouveau type construit par l'utilisateur,
 - Le nom d'une **classe** peut être utilisé en **Java** comme le type d'un attribut ou d'une variable ou aussi comme le type de retour d'une fonction (méthode).
 - Le processus de création d'un **objet** à partir d'une **classe** est appelé en jargon objet **instanciation** d'un objet ou création d' **instance d'une classe**.
 - Exemple : **String** **s1** = "Hello" ;
- Équivalent à : **String** **s1** = **new** String ("Hello") ;
 - **String** est le nom de la classe, la variable **s1** est un **objet instancié** ou **instance de la classe String**.

2. Approche Objet

- Un objet :
 - est un élément d'une classe;
 - a une durée de vie et une identité.



- L'identité d'un objet (**Oid**) :
 - Unique et indépendante de la valeur des propriétés de l'objet;
 - invisible à l'utilisateur.

2. Approche Objet : Classe

- Les **classes** constituent le concept de base de la programmation objet.
- Elles permettent de définir des nouveaux types de données qui doivent se comporter comme des types pré définis et dont les détails d'implantation sont **cachés aux utilisateurs** de ces classes.
- Une définition de classe est introduite par le mot clé **class**.
- Une classe définit généralement deux choses:
 - les **données associées aux objets** de la classe appelés champs ou attribut de la classe,
 - les **services que peuvent rendre les objets** de cette classe interactifs, qui sont les méthodes définies dans la classe.

2. Approche Objet : Classe

Syntaxe générale d'une classe

```
modificateur_visibilité class nom_Classe {  
    modificateur_visibilité data-type attribut1 ;  
    modificateur_visibilité data-type attribut2 ;  
    ...  
    modificateur_visibilité data-type attributN ;  
    modificateur_visibilité Return-type nom_Method1(  
        paramètres)  
    { // instructions ; }  
    ...  
    modificateur_visibilité Return-type nom_Method2(  
        paramètres)  
    { // instructions ; }  
}
```

2. Approche Objet : Classe

- Exemple de classe :

Fichier "Date.java"

```
class Date {  
    private int j;        // jour  
    private int m;        // mois  
    private int a;        // an  
    void initialise(int jour, int mois, int an)  
    { j=jour; m=mois; a=an; }  
    void affiche( )  
    { System.out.println(j + "/" + m + "/" + a); }  
}
```

3. Attributs de classe, attributs d'instance

- Les données **membres** d'une classe sont appelées dans le jargon informatique **attributs**.
- Java distingue deux types **d'attributs** :
 - **Attribut d'instance** dont la valeur peut varier d'une instance à une autre, exemple :
 - La valeur de l'attribut « **Couleur** » de la classe « **Voiture** » varie d'une instance à une autre.
 - **Attribut de classe** dont la valeur est commune pour toutes les instances, exemple :
 - La valeur « **pi** » dans la classe « **Cercle** » est la même 3.14 pour chaque instance de la classe.

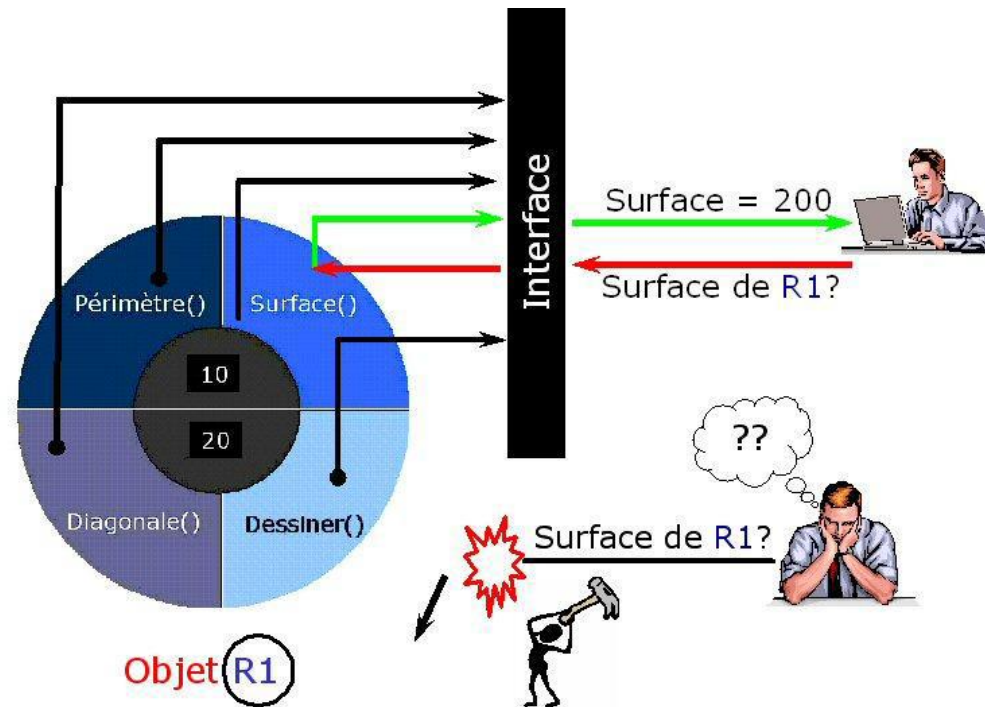
classe : **cercle**
rayon: donnée d'instance
pi: donnée de classe

4. Encapsulation

- L'encapsulation d'un objet par son interface permet de masquer son contenu : Montrer uniquement ce qui est nécessaire pour son utilisation:
 - Les **données** sont généralement considérées comme données **privées**. Elles ne sont pas **accessibles** directement.
 - Les **méthodes** constituent **l'interface d'interaction** avec un **objet** d'une classe. Elles sont donc accessibles (**publiques**).
 - Un objet n'est accessible qu'à travers l'interface de sa classe;
- Java propose plusieurs niveaux de **visibilité** (**modificateur d'accès**) utilisable sur les données d'une classe pour assurer le principe de l'encapsulation.

4. Encapsulation

- Pour respecter ce principe d'**encapsulation**, on ne doit interagir avec un **objet** que par l'**invocation** d'une de ces **méthodes** de son interface.



- On dit à l'objet ce qu'il doit faire, **sans se préoccuper** de comment **il doit le faire**.

5. Accès aux membres : Modificateurs

- Les **membres** sont **accessibles** via une **instance de la classe** ou via **la classe** (pour les membres **statiques**).
- On distingue 4 types d'accès aux membres d'une classe :
 - On accède à **un attribut d'une instance**.
 - On invoque la **méthode d'une instance**.
 - On accède à un **attribut statique** d'une classe
 - On invoque une **méthode statique** d'une classe.
- L'accessibilité des membres est pondérée par des modificateurs de visibilité (**public**, **private**, ...)

5. A. Modificateurs de visibilité

- Valable pour **un attribut ou une méthode** :
 - « **private** » : accessible uniquement par les méthodes de la classe.
 - **rien** : accessible uniquement par les méthodes du même package.
 - « **protected** » :
 - accessible par les méthodes du même package et les méthodes des classes dérivées.
 - « **public** » : accessible par les méthodes de tous les packages.
- Valable pour **une classe** :
 - **rien (friendly)** : accessible uniquement par les classes du même package.
 - « **public** » : accessible par les classes de tous les packages.

5. B. Spécificateurs optionnels

- « **static** » :
 - Attribut statique : sa valeur est commune à toutes les instances
 - Méthode statique : son comportement est commun à toutes les instances.
- « **final** » :
 - Une classe définie comme **final** ne peut pas être dérivée
 - Une méthode déclarée **final** ne peut pas être redéfinie dans une classe dérivée.
 - Un attribut défini comme **final** est une constante (sa valeur ne pourra jamais être modifiée).

6. Constructeur d'une classe

- Un **constructeur** est une méthode **d'initialisation d'un objet** :
 - un **constructeur** n'a **pas** de **type de retour**, et se nomme toujours de la même manière que sa classe,
 - Une classe peut avoir plusieurs constructeurs qui diffèrent par le nombre et la nature de leurs paramètres (principe de **surcharge**).
 - **Par défaut, toute classe récemment créée possède un constructeur sans paramètre.**
- **Utilité** d'un constructeur :
 - un constructeur est une méthode spéciale **appelée à la création** d'un objet **pour initialiser cet objet**.
 - **surcharger** les constructeurs pour une même classe ce qui permet d'initialiser les objets de diverses manières.
- **A noter** : La **surcharge** peut être appliquée pour n'importe quelle méthode et pas seulement pour les constructeurs.

6. Constructeur d'une classe

Fichier " Date.java "

```
class Date {  
    private int j;    // jour  
    private int m;    // mois  
    private int a;    // an  
    void initialise(int jour, int mois, int an)  
    { j=jour; m=mois; a=an; }  
  
    Date( )           // constructeur sans paramètres  
    { j=1; m=1; a=1900; }  
  
    Date(int jour, int mois, int an) // constructeur avec paramètres  
    { j=jour; m=mois; a=an; }  
  
    void affiche( )  
    { System.out.println(j + "/" + m + "/" + a); }  
}
```

Appels de constructeur

- Exemple 1:

- Le constructeur sans arguments initialise la date d1 à la date 1/1/1900.

```
// création d'un objet Date d1 = (1/1/1900)  
Date d1 = new Date();
```

- Exemple 2:

- Le constructeur **Date** à 3 arguments (année, mois, jour) permet d'initialiser une date d2 à une date définie depuis les paramètres.

```
// création d'une date d2 = (01/01/1900)  
Date d2 = new Date(1,1,1900);
```

- Attention :

- Si aucun constructeur n'est spécifié, dans la définition de la classe, un constructeur par défaut vous est obligatoirement fourni, celui-ci n'admettant aucun paramètre.

7. L'attribut caché : this

- Le mot-clé **this** représente une **référence** sur l'objet courant. : **this**.<attribut ou méthode> :
 - celui qui est en train d'exécuter la méthode dans laquelle se trouvent les instructions concernées.
 - **this** sert à lever des **ambiguïtés** entre des **noms d'attributs** et des **noms d'arguments** de méthodes
 - Càd, lorsqu'une variable locale (ou un paramètre) porte le même nom qu'un attribut de la classe.
- **this** peut être utilisé **pour déclencher un constructeur** depuis un autre constructeur : **this** (...)

7. L'attribut caché : this

- Exemple d'utilisation de **this** pour lever des ambiguïtés :

Fichier " Date.java "

```
class Date {  
    private int j;    // jour  
    private int m;    // mois  
    private int a;    // an  
    void initialise(int jour, int mois, int an)  
    { j=jour; m=mois; a=an; }  
    Date( )           // constructeur sans paramètres  
    { j=1; m=1; a=1900; }  
    Date(int j, int m, int a) // constructeur avec paramètres  
    { this.j=j; this.m=m; this.a=a; }  
  
    void affiche( )  
    { System.out.println(j + "/" + m + "/" + a); } }
```

7. L'attribut caché : this

- Exemple d'utilisation de **this** pour déclencher un constructeur depuis un autre constructeur :

Fichier " Date.java "

```
class Date {  
    private int j;    // jour  
    private int m;    // mois  
    private int a;    // an  
  
    Date( )            // constructeur sans paramètres  
    { j=1; m=1; a=1900; }  
  
    Date(int j, int m, int a) // constructeur avec paramètres  
    { this.j = j; this.m = m; this.a = a; }  
  
    void affiche( )  
    { System.out.println(j + "/" + m + "/" + a); }  
}
```

7. L'attribut caché : this

Fichier " Date.java "

```
class Date {  
    private int j;        // jour  
    private int m;        // mois  
    private int a;        // an  
    Date( ) { j=1; m=1; a=1900; }  
    void initialise(int jour, int mois, int an)  
    { j=jour; m=mois; a=a; }  
    Date(int j, int m, int a)        // constructeur avec paramètres  
    { this.j = j; this.m = m; this.a = a; }  
    void affiche( ) { System.out.println(j + "/" + m + "/" + a); }  
    public static void main(String[ ] args)  
    {  
        Date d = new Date(1,6,1997);  
        Dated2= new Date( );  
        d2.affiche( );  
        d.initialise(2,6,1997);  
        d.affiche( );  
    }  
}
```

8. Création d'instances de classes

- Les **instances des classes** (objets) sont systématiquement manipulées à l'aide d'une **référence** (pointeur sur la zone mémoire représentant l'objet).
- la création d'une variable instance d'une classe nécessite deux opérations:

- 1-la création d'une référence qui ne pointe sur rien:

```
Date d1;  
Date d2;
```

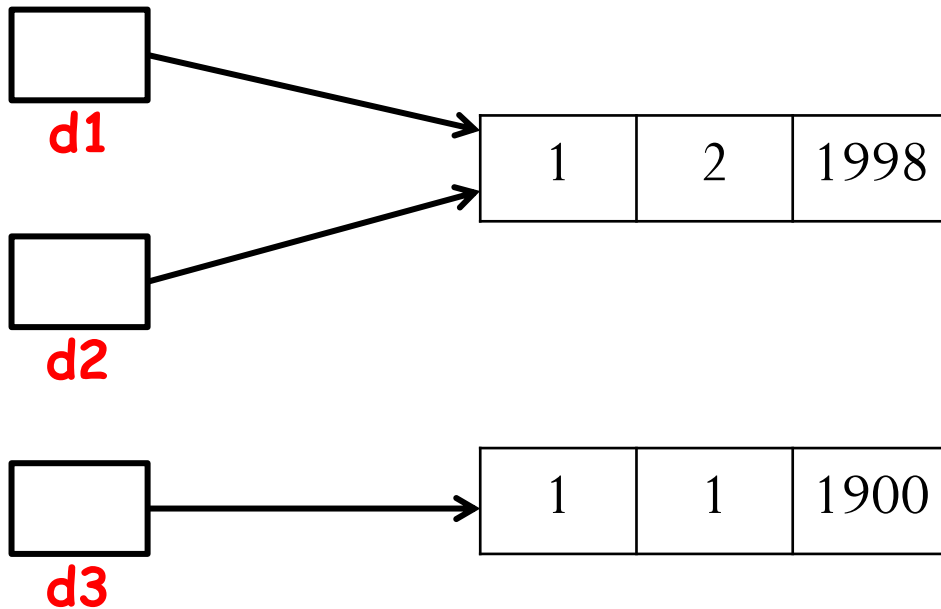
- 2-la réservation de la place mémoire à l'aide de l'opérateur **new** et l'affectation de la donnée.

```
d1 = new Date(); //le constructeur initialise la date courante.
```

```
d2 = new Date("2001/12/11"); //le constructeur initialise la date d2 à  
11/12/2001.
```

8. Création d'instances de classes

- Schéma d'implantation en mémoire :



```
Date d1;
```

```
d1 = new Date(1,2,1998);
```

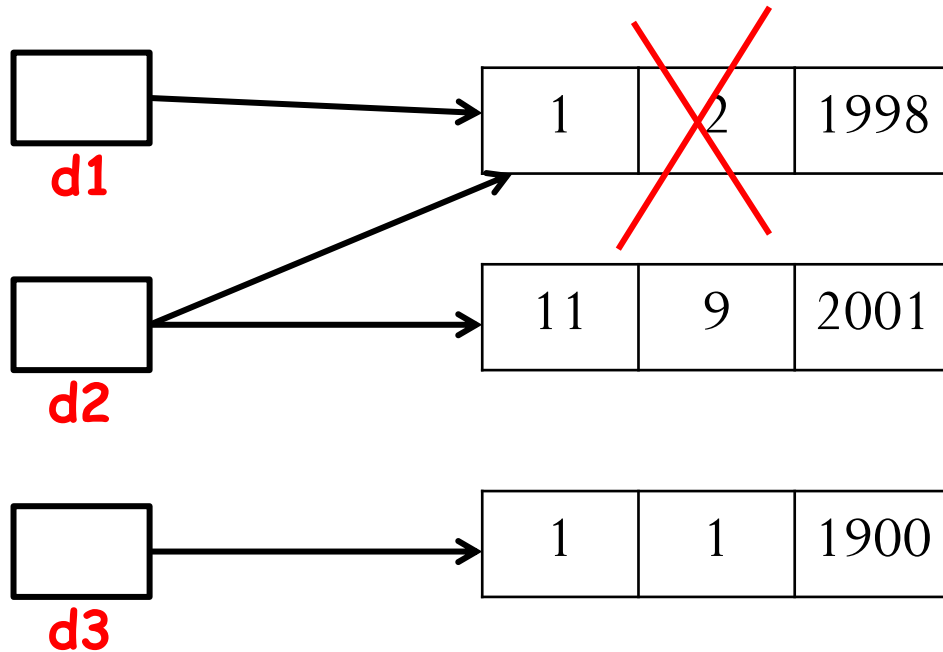
```
Date d2;
```

```
d2 = d1;
```

```
Date d3 = new Date();
```

8. Création d'instances de classes

- Schéma d'implantation en mémoire :



```
Date d1;
```

```
d1 = new Date(1,2,1998);
```

```
Date d2;
```

```
d2 = d1;
```

```
Date d3 = new Date();
```

```
d1= null;
```

```
d2 = new Date(11,9,2001);
```

9. Destruction d'un objet

- La **plupart des langages orientés objet** possèdent une méthode nommée **destructeur**, appelée pour **détruire un objet**.
- La **destruction** d'un objet consiste à **libérer les ressources** qui lui ont été déjà affectées par le constructeur .
- Un objet est **détruit** lorsqu'il **ne référence aucune zone mémoire**.
- En **Java**, la **destruction** des objets est prise en charge par un mécanisme automatique appelé **ramasse-miettes** (**garbage collector GC**).
- Le **GC** fonctionne en arrière plan pendant l'exécution d'un programme **Java**. Il est géré dans un **thread** de faible priorité.
- difficile de savoir à quel moment le **GC** détruira réellement votre objet.

9. Destruction d'un objet

- La récupération de mémoire peut être aussi invoquée explicitement par le programmeur à des moments bien précis avec la commande **System.gc();**
- Certaines situations nécessitent un nettoyage spécial que le **GC** ne peut pas effectuer :
 - **Par exemple**, certains **fichiers** ont été **ouverts** pendant la durée de **vie de l'objet** et vous voulez **vérifier** qu'ils sont correctement fermés **quand l'objet est détruit**;
 - Pour cela, une autre méthode spéciale, appelée **finalize()**, peut être définie;
 - Cette méthode (si elle est présente) est **appelée** par le **GC** lorsque l'objet est détruit pour assurer ce type de nettoyage spécial.

Fichier " Date.java "

```
class Date {  
    private int j;        // jour  
    private int m;        // mois  
    private int a;        // an  
  
    void initialise(int jour, int mois, int an) { j=jour; m=mois; a=an; }  
    Date( ) { j=1; m=1; a=1900; }  
    Date(int j, int m, int a) { this.j = j; this.m = m; this.a = a; }  
    void affiche( ) { System.out.println(j + "/" + m + "/" + a); }  
  
    public void finalize() { System.out.println("Nettoyage spécial"); }  
  
    public static void main(String[ ] args)  
    {  
        Date d = new Date(1,6,1997);  
        d.affiche( );  
        d=null;           // d ne désigne plus rien, il ne possède plus de référence.  
                           // Elle n'est plus accessible. A tout moment le GC peut détruite l'objet.  
        System.gc( );     // appel implicite de finalize  
    }
```

Résultat d'affichage

1/6/1997

Nettoyage spécial

Fichier " Date.java "

```
class Date {  
    private int j;        // jour  
    private int m;        // mois  
    private int a;        // an  
  
    void initialise(int jour, int mois, int an) { j=jour; m=mois; a=an; }  
    Date( ) { j=1; m=1; a=1900; }  
    Date(int j, int m, int a) { this.j = j; this.m = m; this.a = a; }  
    void affiche( ) { System.out.println(j + "/" + m + "/" + a); }  
  
    public void finalize() { System.out.println("Nettoyage spécial"); }  
  
    public static void main(String[ ] args)  
    {  
        Date d = new Date(1,6,1997);  
        d.affiche( );  
        Date d2=d;  
        d =null;  
        System.gc( );  
    }  
}
```

Résultat d'affichage

1/6/1997

Fichier " Date.java "

```
class Date {  
    private int j;        // jour  
    private int m;        // mois  
    private int a;        // an  
  
    void initialise(int jour, int mois, int an) { j=jour; m=mois; a=an; }  
    Date( ) { j=1; m=1; a=1900; }  
    Date(int j, int m, int a) { this.j = j; this.m = m; this.a = a; }  
    void affiche( ) { System.out.println(j + "/" + m + "/" + a); }  
  
    public void finalize() { System.out.println("Nettoyage spécial"); }  
  
    public static void main(String[ ] args)  
    {  
        Date d = new Date(1,6,1997);  
        d.affiche( );  
        Date d2=d;  
        d=null; d2=null;  
        System.gc( );  
    }  
}
```

Résultat d'affichage

1/6/1997

Nettoyage spécial

10. Comparaison d'instances

- Soit en utilisant l'opérateur « == »:
- Il y'a égalité que si d1 et d2 **désignent la même zone mémoire** représentant une date.

```
Date d1 = new Date(1,2,2000);
```

```
Date d2 = new Date(1,2,2000);
```

```
if (d1==d2) . . . // il n'y a pas égalité
```

```
Date d1 = new Date(1,2,2000);
```

```
Date d2 = d1;
```

```
if(d1==d2) . . . // il y a égalité
```

- Soit en utilisant la méthode « equals » :

```
Date d1 = new Date(1,2,2000);
```

```
Date d2 = new Date(1,2,2000);
```

```
if(d1.equals(d2)) . . . // il y a égalité si « equals » est redéfinie.
```

11. Modes de passage

- Le **mode de passage** des paramètres dans les méthodes dépend de la nature des paramètres :
 - **par référence pour les objets,**
 - **par copie pour les types primitifs.**

```
class Test
{
    public String s;
    public int i;
    Test() { };           // constructeur
    void modifier(int value, StringBuffer ch)
    { value++; ch.append(" tout le monde"); }
    public static void main(String[] args)
    {
        Test t = new Test();
        int a=0; StringBuffer chaine= new StringBuffer("Bonjour");
        t.modifier(a, chaine);
        System.out.println(" La valeur de a="+a);
        System.out.println(" La valeur de s="+chaine);
    }
}
```

Résultat d'affichage

La valeur de a = 0

La valeur de s= Bonjour tout le monde

12. Les opérateurs sur les références

- Les **seuls opérateurs sur les références** sont des opérateurs logiques :
 - **==**: permet de tester si deux références désignent le même objet.
 - **!=**: permet de tester si deux références ne désignent pas le même objet.
 - **equals**: permet de comparer le contenu de deux références.
 - **instanceof**: permet de tester si l'objet référencé est une instance d'une classe donnée ou d'une de ses sous-classes

```
if ( d1 instanceof Date ) {  
    ... Traitement  
}
```